

❏ 1. Introduction to Performance Tuning

❏ What is Performance Tuning?

Performance tuning is the **process of optimizing system performance** by adjusting and configuring hardware and software parameters to achieve optimal efficiency and responsiveness.

❏ Goals of Performance Tuning:

- Reduce latency
- Maximize throughput
- Efficient resource utilization
- Improve system stability
-

Reduce downtime

⌘ Key Areas of Tuning:

Area	Tuning Focus
CPU	Load balancing, interrupt handling, processor affinity
Memory (RAM)	Swap usage, buffer/cache management
Disk I/O	Filesystem type, RAID config, block size
Network	Bandwidth, packet loss, latency, TCP stack tuning
Applications	Threading, caching, code optimization

⌘ Tools for Performance Monitoring & Tuning:

Tool	Use
top, htop	CPU/memory usage
vmstat	Virtual memory statistics
iotop, iostat	Disk I/O stats
netstat, ss	Network connections
perf	Linux performance counters
sysctl	Tuning kernel parameters

⌘ Kernel Tuning (Example):

bash

Copy code

```
sysctl -w vm.swappiness=10
```

To make changes persistent:

```
bash
```

Copy code

```
echo "vm.swappiness=10" >> /etc/sysctl.conf
```

❏ 2. Maintenance and Troubleshooting

❏ What is System Maintenance?

Regular activities performed to ensure systems are secure, up-to-date, stable, and optimized.

☒ Types of Maintenance:

- **Preventive** – Backups, patches, monitoring.
 - **Corrective** – Fix issues like failing services.
 - **Adaptive** – Adjusting for new needs (e.g., scaling).
 - **Perfective** – Improving performance/usability.
-

☒ Common Maintenance Tasks:

- Log rotation (logrotate)
- Patch management (yum update, apt update)

- Disk cleanup (journalctl --vacuum-time, rm logs)
 - Service restarts (systemctl restart)
-

☒ Troubleshooting Approach:

1.

Identify the Problem

- Symptoms? Logs?

2.

Isolate the Cause

- Use logs (/var/log/syslog, journalctl)

- Check recent changes

3.
Implement a Fix

- Patch or reconfigure

4.
Test the Fix

5.
Document the Issue & Solution

☒ Useful Troubleshooting Commands:

Command	Purpose
dmesg	Kernel messages
journalctl	Systemd log viewer
systemctl status	Check service status
ping, traceroute	Network diagnostics
df, du	Disk usage
ps, top	Process status

❏ 3. The Threat Model and Protection Methods

❏ What is a Threat Model?

A **threat model** is a structured representation of potential threats to a system, outlining attack surfaces, vulnerabilities, and defenses.

☒ Why Threat Modeling?

- Understand **what assets** need protection.
 - Determine **who the adversaries** are.
 - Identify **how they might attack**.
 - Plan **how to prevent or mitigate** such attacks.
-

☒ Elements of a Threat Model:

1. **Assets** – What needs protection (data, systems)
2. **Adversaries** – Who is attacking (external, insider)

3.

Entry Points – Where attackers might enter (ports, APIs)

4.

Attack Vectors – How attackers exploit vulnerabilities

5.

Countermeasures – Controls to mitigate threats

☒ Common Threat Modeling Frameworks:

-

STRIDE (Microsoft):

-

Sp spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege

-

DREAD (Risk Assessment):

- Damage, Reproducibility, Exploitability, Affected users, Discoverability

☒ Protection Methods:

Area	Methods
Authentication	Strong passwords, MFA
Authorization	Role-based access control (RBAC)
Data Security	Encryption (TLS, GPG), backups
Network Security	Firewalls, IDS/IPS, VPN
Endpoint Security	Antivirus, patch management
Physical Security	Locks, surveillance
Monitoring	SIEM, audit logs